

Concise One-Layer Transformers Can Perform Dictionary Lookup

Anton Hampe

Lena Strobl

Dana Angluin

Robert Frank

Yale University, New Haven, CT, USA

ANTON.HAMPE@YALE.EDU

LENA.STROBL@YALE.EDU

DANA.ANGLUIN@YALE.EDU

ROBERT.FRANK@YALE.EDU

Abstract

While transformers have proven enormously successful in a range of tasks, their fundamental properties as models of computation are not well understood. This paper contributes to the study of the expressive capacity of transformers, focusing on their ability to perform the fundamental computational task of looking up a key in a dictionary. We prove that concise 1-layer transformers (i.e., with a polylog bound on the product of the number of heads, the embedding dimension, and precision) can perform this task when the key and value are represented in the same input position, but, under the p -bit rational precision assumption, cannot perform it when the keys and values are represented in different input positions. Concise 2-layer transformers can perform the task in the second case. Experimentally, we find an alignment between what we have proven can be computed by concise transformers and what can be practically learned.

Keywords: transformers, expressivity, dictionary lookup, attention, communication complexity

1. Introduction

The transformer architecture for neural networks has had a revolutionary impact on the development and deployment of large language models. There are many papers with empirical assessments of the performance of transformers for various tasks, but less is known about their in-principle limits as models of computation.

Expressivity and Trainability. This paper, like most current theoretical work, addresses *expressivity*, that is, whether there exists any set of weights that would allow a transformer to perform a given task. Expressivity is a pre-condition of *trainability*, that is, whether there is an optimization procedure that could efficiently find some set of weights that would allow a transformer to perform a given task, given examples of it. Of course, expressivity does not ensure trainability. For example, [Chiang and Cholak \(2022\)](#) show that a multi-layer transformer encoder can compute the parity of a binary string, but to date, successful training has required carefully crafted multi-stage curricula ([Kim and Suzuki, 2025](#)). Corresponding to our theoretical results on expressivity, we provide experimental results illustrating the trainability of the task we consider under various conditions.

Universality and Conciseness. Expressivity has been addressed by universality results. Yun et al. (2020) prove that an arbitrary function can be approximated by a transformer, although their construction requires unbounded depth. Kajitsuka and Sato (2024) strengthen this result, showing that transformers with a single self-attention layer are universal function approximators, however, in general their construction requires a transformer of size exponential in the input length. In these constructions, the transformer depends on the input length, and we refer to this situation as a *family of transformers*, by analogy to a family of circuits. We view the growth rate of these general solutions as impractically large, and seek *concise* solutions. We characterize transformer size in terms of their *c-size*: the product of the embedding dimension, the number of heads, and precision. We consider concise transformers to be those whose c-size grows as a polylogarithmic function of its input length. We study the expressive power of a concise family of one-layer transformers with a single head as the basis for more complex architectures with multiple heads, multiple layers, looping, chain of thought, diffusion, and other enhancements.

Dictionary lookup. The task we consider is *dictionary lookup* (or *associative lookup*) in which the input is a dictionary consisting of a finite list of key/value pairs together with a target key, and the desired output is the value that is paired with the target key in the dictionary. This task is a fundamental building block of many algorithms, and understanding the conditions under which it can be performed by a transformer will help us understand whether and how transformers can perform more complex algorithms.

The keys and values are tokens from a vocabulary $\mathcal{V}$ with $|\mathcal{V}| = N$, and there is a token embedding that maps each token to a vector. We study two different input representations of a dictionary.

Same position. The two vectors representing a key/value pair are concatenated and placed in one input position.

Consecutive positions. The two vectors representing a key/value pair are placed in consecutive input positions.

In the same position case, each key/value association is local to one input position, so one attention layer can retrieve the corresponding value directly. In the consecutive positions case, each association is distributed across two input positions.

Our Contributions.

- For the same position case, in §3 we construct a concise family of 1-layer hard attention transformers with a single head, embedding dimension $O(1)$ and $O(\log N)$ bits of precision that can perform dictionary lookup. We show in §4 that this construction is effective in transformers with soft attention with the addition of temperature.
- For the consecutive positions case, in §5 we prove (non-concise) lower and upper bounds of $\Theta(N \log N)$ on the c-size of 1-layer softmax attention transformers that can perform dictionary lookup. We also construct a concise family of 2-layer hard attention transformers with one head, embedding dimension $O(1)$ and $O(\log N)$ bits of precision that can perform dictionary lookup.

- For the same position case, in §6 we find that randomly initialized small transformers can be trained with gradient descent to perform dictionary lookup, though at embedding dimensions larger than in the theoretical construction. For the consecutive positions case, we find that 1-layer transformers struggle to learn dictionary lookup, even for small vocabularies and dictionaries.

Related work. Strobl et al. (2024) survey theoretical results on the expressiveness of transformers as models for the recognition of formal languages. Yang et al. (2025) give a compendium of theoretical techniques useful for constructing transformers to perform various tasks. Our work also builds on previous theoretical work showing lower bounds on 1-layer transformers for certain tasks.

Communication complexity has been a fruitful approach to lower bounds. Peng et al. (2024) consider the task of function composition and use communication complexity to demonstrate that transformers with a single attention layer of fixed size are unable to perform function composition over functions of sufficiently large domain. Sanford et al. (2024) focus on the capacity of 1-layer transformers to perform the induction head task (Elhage et al., 2021; Olsson et al., 2022). They use communication complexity to prove a lower bound of $\Omega(n)$ on the transformer size necessary to perform this task, where n is the input length.

An alternative approach using VC-dimension to show lower bounds on 1-layer transformers is presented by Kozachinskiy et al. (2025). They prove a lower bound $\max\{h, d, P\} \geq n^{\Omega(1)}$ on 1-layer transformers to compute function and relation composition, where h is the number of heads, d is the embedding dimension, and P is the number of parameters in the feedforward layer.

2. Preliminaries

2.1. Conventions

For any positive integer n , $[n]$ denotes the set of integers $\{0, 1, \dots, n-1\}$. We use the notation $(\mathbb{R}^d)^+$ for sequences (of arbitrary finite positive length) with elements from $\mathbb{R}^d$. If $\mathbf{X} \in (\mathbb{R}^d)^+$ has length n , then for $i \in [n]$, $\mathbf{X}[i] \in \mathbb{R}^d$ refers to element i of the sequence, with 0-based indexing. A function $g: (\mathbb{R}^d)^+ \rightarrow (\mathbb{R}^d)^+$ is *length-preserving* if it maps an input sequence of length n to an output sequence of the same length n .

We consider an extension of softmax to softmax_T to incorporate a *temperature* T , where the effect of lower temperature is to magnify the differences in attention weights for two different attention scores. Standard softmax then corresponds to temperature $T = 1.0$. For $\mathbf{x} \in \mathbb{R}^n$ and real-valued $T > 0$,

$$\text{softmax}_T(\mathbf{x})[i] = \frac{\exp(\mathbf{x}[i]/T)}{\sum_{j \in [n]} \exp(\mathbf{x}[j]/T)}.$$

2.2. Dictionary Lookup

For the dictionary lookup task, there is a fixed vocabulary $\mathcal{V}$ of tokens, where $N = |\mathcal{V}|$. We assume an arbitrary ordering $\tau_0, \tau_1, \dots, \tau_{N-1}$ of the elements of $\mathcal{V}$, and a function idx such that $\text{idx}(\tau_j) = j$. This ordering is used in the matrices that embed and unembed tokens. A *dictionary* of length n is a finite sequence of key/value pairs $(k_0, v_0), \dots, (k_{n-1}, v_{n-1})$ where

$(k_i, v_i) \in \mathcal{V} \times \mathcal{V}$ for $i \in [n]$, such that no two pairs have the same key, that is, if $i \neq j$ then $k_i \neq k_j$. Note that we must have $n \leq N$.

A *dictionary lookup problem* consists of a dictionary and a *target key* $k^* \in \mathcal{V}$, a token such that for some i , $k^* = k_i$. The correct output for a dictionary lookup problem is the unique token v_i such that $k^* = k_i$. Note that the correct output does not depend on the order of pairs in the dictionary. For ease of understanding, we leave aside the variant of dictionary lookup in which the target key does not occur in the dictionary, though it should not be a difficult extension.

The dictionary lookup task is similar to the induction heads task considered by [Sanford et al. \(2024\)](#). In their definition of the standard induction heads task, the input is a sequence of symbols from a fixed vocabulary, and the output at each position is the symbol that follows the most recent previous occurrence of the symbol at that position, or a null symbol if there is no previous occurrence. For example, for $X = \text{baebcabebdea}$, the final symbol is a , which occurs twice previously, and the final output is b , the symbol following its most recent previous occurrence. In this task, there is no distinction between keys and values, and there may be multiple occurrences of the symbol being sought.

An important difference is that in the induction heads task, the vocabulary size is a constant independent of the input length, and constructions typically use a one-hot encoding of symbols. In the dictionary lookup task, the vocabulary may be as large as the input length, which emphasizes the properties of the vocabulary embedding. In particular, a one-hot encoding of the vocabulary cannot yield a concise family of transformers for dictionary lookup.

2.3. Transformer Models

We follow the definition of a transformer model given by [Yang et al. \(2025\)](#). A *transformer* of embedding dimension d is a mapping from a non-empty input sequence to an output. It has a length-preserving *input embedding*, which maps a sequence of inputs to an element of $(\mathbb{R}^d)^+$, a sequence of length-preserving attention and feedforward layers that map $(\mathbb{R}^d)^+$ to $(\mathbb{R}^d)^+$, and an *output mapping*, which maps from $(\mathbb{R}^d)^+$ to an output, which in our case is a single element of $\mathbb{R}^d$.

Input Embedding. We specialize the input embedding for the two input presentations we consider.

For the consecutive positions case, the input alphabet is $\mathcal{V}$ (a key or value), and we assume that the input order is $k_0, v_0, \dots, k_{n-1}, v_{n-1}, k^*$. The input length $\ell(n)$ is $2n + 1$. The *word embedding* $\mathbf{w}$ maps $\mathcal{V}$ to $\mathbb{R}^d$, and the *position embedding* maps $[\ell(n)]$ to $\mathbb{R}^d$. The *input embedding* $\mathbf{e}$ maps a sequence of input tokens $x_0, \dots, x_{\ell(n)-1}$ to a sequence of elements of $\mathbb{R}^d$ by $\mathbf{e}(x_i) = \mathbf{w}(x_i) + \mathbf{p}(i)$. This presentation is typical when considering dictionary lookup as an end-to-end task.

For the same position case, the input alphabet of the transformer is $(\mathcal{V} \times \mathcal{V}) \cup \mathcal{V}$ (a key/value pair or a key), and we assume the order is $(k_0, v_0), \dots, (k_{n-1}, v_{n-1}), k^*$. The input length $\ell(n)$ is $n + 1$. In this case, we assume an embedding $\mathbf{e}_{\text{tok}}$ that maps tokens from $\mathcal{V}$ to $\mathbb{R}^{d_{\text{tok}}}$. The embedding dimension is $d = 2d_{\text{tok}}$, and the *word embedding* $\mathbf{w}$ maps $(k_i, v_i) \in \mathcal{V} \times \mathcal{V}$ to $\mathbb{R}^d$ by concatenating the vectors $\mathbf{e}_{\text{tok}}(k_i)$ and $\mathbf{e}_{\text{tok}}(v_i)$. The word embedding $\mathbf{w}$ maps the target key k^* to the concatenation of $\mathbf{e}_{\text{tok}}(k^*)$ and $\mathbf{0} \in \mathbb{R}^{d_{\text{tok}}}$. In this case, there

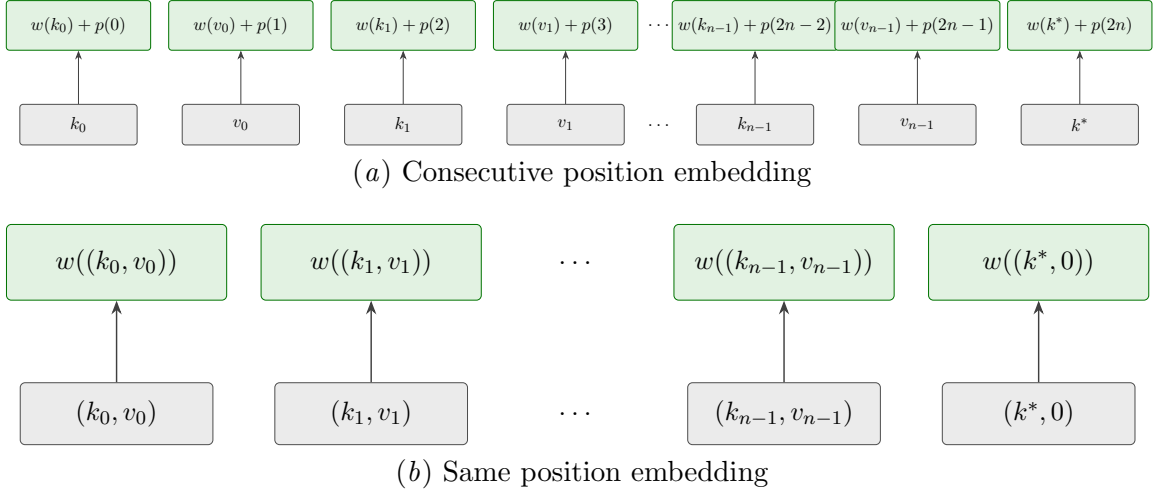


Figure 1: Comparison of input embedding methods

is no position embedding, so the input embedding $\mathbf{e}$ is just $\mathbf{w}$. This presentation is intended to reflect a situation in which earlier layers of the transformer have constructed a favorable representation of the task. This is analogous to the second layer of a 2-layer construction to perform the induction heads task, in which the first layer has copied into each position the symbol at the preceding position (Sanford et al., 2024). See also Sec. §5.3.

Scaled Dot-Product Softmax Attention Layer. We specialize our definition to one attention head per layer. A (*scaled dot-product*) *softmax self-attention layer* with d input/output dimensions and d_{key} key/value dimensions is a length-preserving function mapping $\mathbf{X} \in (\mathbb{R}^d)^+$ to $\mathbf{Y} \in (\mathbb{R}^d)^+$ parameterized by $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d_{\text{key}} \times d}$ and $\mathbf{W}^V \in \mathbb{R}^{d \times d}$. For positions i and j in $\mathbf{X}$ and $\mathbf{Y}$, the *query* at i is $\mathbf{Q}[i] = \mathbf{W}^Q(\mathbf{X}[i])$, the *key* at j is $\mathbf{K}[j] = \mathbf{W}^K(\mathbf{X}[j])$, and the *value* at j is $\mathbf{V}[j] = \mathbf{W}^V(\mathbf{X}[j])$. Then we define:

$$\begin{aligned} \mathbf{s}[i, j] &= \frac{\mathbf{Q}[i] \cdot \mathbf{K}[j]}{\sqrt{d_{\text{key}}}} \\ \alpha[i, :] &= \text{softmax}(\mathbf{s}[i, :]) \\ \mathbf{Y}[i] &= \sum_{j \in [n]} \alpha[i, j] \mathbf{V}[j] \end{aligned}$$

We call $\mathbf{s}[i, j]$ the *attention scores* and $\alpha[i, j]$ the *attention weights*.

To include *residuals* and *pre-norm layer normalization* we would instead set $\mathbf{Y}[i] = \mathbf{X}[i] + \text{LN}(\sum_{j \in [n]} \alpha[i, j] \mathbf{V}[j])$, where LN is a layer normalization function with parameters $\gamma, \beta \in \mathbb{R}^d$. In Section 4 we also consider using softmax_T , softmax with temperature T (§2.1), in place of standard softmax.

Average Hard Attention. For simplicity, our theoretical constructions use average hard attention instead of softmax attention; we address this difference in our empirical results.

For any vector $\mathbf{x} \in \mathbb{R}^n$, define $M(\mathbf{x}) = \{i \in [n] \mid \forall j \in [n], \mathbf{x}[j] \leq \mathbf{x}[i]\}$ to be the set of indices of the maximal elements of $\mathbf{x}$. In an *average hard* attention layer, instead of the softmax computation, the elements of $M(\mathbf{s}[i, :])$ are each assigned attention weight $1/|M(\mathbf{s}[i, :])|$ and the other elements are assigned attention weight 0. Note that if there is a unique position j with the maximum attention score, it receives weight 1 and all other positions receive weight 0, which enables exact copying of $\mathbf{V}[j]$.

Feedforward Layers. A *feedforward layer* is a mapping f from $\mathbb{R}^d$ to $\mathbb{R}^d$ that can be implemented by a 2-layer multilayer perceptron with ReLU activation. It is determined by a dimension d_{hid} and two affine transformations, one from $\mathbb{R}^d$ to $\mathbb{R}^{d_{\text{hid}}}$ and one from $\mathbb{R}^{d_{\text{hid}}}$ to $\mathbb{R}^d$. The input $\mathbf{X}$ is mapped to the output $\mathbf{Y}$ elementwise, that is, $\mathbf{Y}[i] = f(\mathbf{X}[i])$. A feedforward layer may also have residuals and pre-norm layer normalization.

Output Mapping. In our case, the desired output is an element of $\mathcal{V}$. Let $\mathbf{Y} \in (\mathbb{R}^d)^{\ell(n)}$ be the output of the final layer of the transformer, and $\mathbf{Y}[\ell(n) - 1]$ its final element. There is an *unembedding matrix* $\mathbf{U} \in \mathbb{R}^{N \times d}$, and the output of the transformer is taken to be the token $\tau_i \in \mathcal{V}$ for the index $i \in [N]$ that maximizes $(\mathbf{U}\mathbf{Y}[\ell(n) - 1])[i]$.

We are primarily concerned with the number of attention layers a transformer has, so in this paper a *k-layer transformer* is one with k attention layers and any constant number of feedforward layers. We consider encoder-only transformers with no masking; this is equivalent to future masking for 1-layer transformers in which the output depends only on the last position.

2.4. The Communication Size of Transformers

As noted in the introduction, we consider *families of transformers*: for every positive integer n , there is a transformer T_n that processes input sequences of length at most n , and we analyze how the parameters of the transformers change as n increases.

We define a transformer’s *communication-size*, henceforth *c-size*, to be the product hdp , where h is the number of heads per layer, d is the embedding dimension, and p is the precision (in bits) of the representation of numbers in the transformer. The notion of c-size reflects the cost of communication, and does not depend on the number of parameters in any feedforward layer. We present upper and lower bounds on the c-size of 1-layer and 2-layer transformers to perform dictionary lookup, expressed as a function of the maximum input length for the transformer.

Sanford et al. (2023), Chen et al. (2024), and Kozachinskiy et al. (2025) distinguish between families of transformers of “small” ($n^{o(1)}$, subpolynomial) and “large” ($n^{\Omega(1)}$, polynomial or more) size.¹ Here we adopt a more stringent notion of conciseness; we say that a family of transformers is *concise* if its c-size is $O(\log^{O(1)} n)$ (poly-logarithmic) in the maximum input length n .

1. Kozachinskiy et al. (2025) define the size of a 1-layer transformer as $\max\{h, d, P\}$, where h is the number of heads, d is the embedding dimension, and P is the number of parameters in the feedforward layer. Thus c-size includes precision but excludes feedforward parameter count, whereas their measure includes feedforward parameter count but does not explicitly include precision.

3. Same Position: A Concise Family of 1-Layer Transformers

In this section we give a construction of a concise family of 1-layer hard attention transformers for dictionary lookup for the same position case. We first define the representation of integers we use.

Representing Integers. For any positive integer m we consider a representation of elements of $[m]$ using angles in a unit circle. For $j \in [m]$, define the angle $\theta_m(j) = 2\pi j/m$, and represent j by the 2-element vector $\text{cs}_m(j) = [\cos(\theta_m(j)), \sin(\theta_m(j))]^\top$. In this representation, for $j, k \in [m]$ the dot product of $\text{cs}_m(j)$ with $\text{cs}_m(k)$ is 1 if $j = k$ and is at most $1 - 1/m^2$ if $j \neq k$. Thus, $O(\log m)$ bits of precision suffice to distinguish two different integers from $[m]$ in this representation. (See Fig. 4 in §A for an example.)

We generalize this representation to include a frequency $f \in [m]$ as follows. Let $\theta_{m,f}(j)$ be the angle $2\pi f j/m$, and define $\text{cs}_{m,f}(j) = [\cos(\theta_{m,f}(j)), \sin(\theta_{m,f}(j))]^\top$. Then $\text{cs}_{m,1}$ is the same as cs_m . In Sections 4 and 6, we consider representing $j \in [m]$ using a concatenation of vectors $\text{cs}_{m,f}(j)$ for multiple frequencies f .

Theorem 1 *For any finite vocabulary $\mathcal{V}$ of cardinality N , there exists a 1-layer transformer that performs dictionary lookup for vocabulary $\mathcal{V}$ in the same position case. The transformer uses average hard attention, one head, embedding dimension 4, $O(\log N)$ bits of precision, no position embedding, no residuals, no layernorm, and no feedforward layer. The c-size is $O(\log N)$.*

Proof The input is a dictionary problem $(k_0, v_0), (k_1, v_1), \dots, (k_{n-1}, v_{n-1}), k^*$, where the keys k_i , values v_i , and target key k^* are elements of $\mathcal{V}$. The desired output is v_j such that $k_j = k^*$.

We construct the input embedding $\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n-1}, \mathbf{x}_n]$, as follows. For $j \in [n]$:

$$\mathbf{x}_j = \begin{bmatrix} \text{cs}_N(\text{idx}(k_j)) \\ \text{cs}_N(\text{idx}(v_j)) \end{bmatrix} = \begin{bmatrix} \cos(\theta_N(\text{idx}(k_j))) \\ \sin(\theta_N(\text{idx}(k_j))) \\ \cos(\theta_N(\text{idx}(v_j))) \\ \sin(\theta_N(\text{idx}(v_j))) \end{bmatrix}, \text{ and}$$

$$\mathbf{x}_n = \begin{bmatrix} \text{cs}_N(\text{idx}(k^*)) \\ \mathbf{0}_{2 \times 1} \end{bmatrix} = \begin{bmatrix} \cos(\theta_N(\text{idx}(k^*))) \\ \sin(\theta_N(\text{idx}(k^*))) \\ 0 \\ 0 \end{bmatrix}.$$

Thus the embedding dimension is 4.

We define the matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$ as:

$$\mathbf{W}^Q = \mathbf{W}^K = \begin{bmatrix} \mathbf{I}_2 & \mathbf{0}_{2 \times 2} \end{bmatrix}, \quad \mathbf{W}^V = \begin{bmatrix} \mathbf{0}_{2 \times 2} & \mathbf{0}_{2 \times 2} \\ \mathbf{0}_{2 \times 2} & 2 * \mathbf{I}_2 \end{bmatrix},$$

where $\mathbf{I}_2$ is the 2×2 identity matrix. For a vector $\mathbf{x} \in \mathbb{R}^4$, each of $\mathbf{W}^Q \mathbf{x}$ and $\mathbf{W}^K \mathbf{x}$ are both the vector of the first two coordinates of $\mathbf{x}$, and $\mathbf{W}^V \mathbf{x}$ is the vector of twice the last two coordinates of $\mathbf{x}$ preceded by two 0's.

For the target key $k^* \in \mathcal{V}$, the query vector at position n is:

$$\mathbf{q}_n = \mathbf{W}^Q \mathbf{x}_n = \text{cs}_N(\text{idx}(k^*)) = [\cos(\theta_N(\text{idx}(k^*))), \sin(\theta_N(\text{idx}(k^*)))]^\top.$$

For each position $j \in [n]$, the key vector is

$$\mathbf{k}_j = \mathbf{W}^K \mathbf{x}_j = \text{cs}_N(\text{idx}(k_j)).$$

At position n , the key vector is

$$\mathbf{k}_n = \mathbf{W}^K \mathbf{x}_n = \text{cs}_N(\text{idx}(k^*)).$$

At position n , the attention scores are

$$s_j = \mathbf{s}[n, j] = \frac{\mathbf{q}_n^\top \mathbf{k}_j}{\sqrt{d_{\text{key}}}}.$$

We have $s_j = 1/\sqrt{2}$ for those j such that $k_j = k^*$. There are two such positions: the $j \in [n]$ that holds the dictionary entry (k_j, v_j) , and position n itself. All other j have $s_j < 1/\sqrt{2}$. Using average hard attention, the two positions j with maximum attention score each get attention weight $1/2$, and all other positions get attention weight 0. The output of the attention layer at position n is:

$$\mathbf{h} = \sum_{j=0}^n \alpha_j \mathbf{W}^V \mathbf{x}_j.$$

This is $[0, 0, \cos(\theta_N(\text{idx}(v_j))), \sin(\theta_N(\text{idx}(v_j)))]^\top$ such that $j \in [n]$ and $k_j = k^*$.

We define the unembedding matrix $\mathbf{U} \in \mathbb{R}^{N \times 4}$ to consist of the N rows $[0, 0, \cos(\theta_N(r)), \sin(\theta_N(r))]$ for $r \in [N]$. Applying it to $\mathbf{h}$, the attention output at position n , we have $\hat{\mathbf{v}} = \mathbf{U}\mathbf{h}$. The final output of the transformer is the index r where $\hat{\mathbf{v}}$ is maximized, which is $r = \text{idx}(v_j)$ such for the $j \in [n]$ such that $k_j = k^*$, that is, the correct output for dictionary lookup. $\blacksquare$

An example of the construction in the proof of [Thm. 1](#) is given in [§B](#).

It might be objected that the transformer constructed in the proof of [Thm. 1](#) is too simplified to show that practical transformers can perform the task of dictionary lookup because it uses average hard attention instead of softmax attention and lacks features like residuals, layernorms, and a feedforward layer. Using techniques described by [Yang et al. \(2025\)](#), we can add residuals, layernorms, and a feedforward layer to the model in such a way that the same output is obtained.²

Corollary 2 *For any finite vocabulary $\mathcal{V}$ of cardinality N , there exists a 1-layer transformer that performs dictionary lookup for vocabulary $\mathcal{V}$ in the same position case. The transformer has average hard attention, no position embedding, residuals, pre-norm layernorms, a feedforward layer, one head, embedding dimension $O(1)$, and $O(\log N)$ bits of precision. The c-size is $O(\log N)$.*

2. These techniques may multiply the embedding dimension by a constant, which leaves our conciseness results unaffected.

For the difference between softmax attention and average hard attention, we could replace average hard attention by softmax attention with temperature. It is well known that softmax attention with temperature approaches average hard attention as the temperature approaches 0. [Yang et al. \(2026\)](#) give quantitative bounds on the rate of approach.

Additional questions of whether practical 1-layer transformers can perform and learn dictionary lookup are addressed in Sections 4 and 6.

4. Simulating the Theoretical Construction with Temperature

The theoretical results for the same position task in Section 3 assume average hard attention, a common technique for proving transformer expressiveness. [Yang et al. \(2026\)](#) show that such proofs for hard attention carry over to transformers with soft attention if a temperature parameter is included in the softmax. We experimentally determine the temperature required for the theoretical construction to achieve 100% accuracy when using soft attention instead of average hard attention.

4.1. Embeddings with Multiple Frequencies

Section 3 introduces a vector representation of positive integers in $[m]$ computed using a vector of ℓ frequencies $f \in [m]^\ell$. In the proofs in Section 3, we limit ourselves to the case of using the single frequency $f = 1$. Here, we consider the more general case of using multiple frequencies and the effect of the choice of the frequency vector on the temperature required to achieve 100% accuracy on the dictionary lookup task using the construction we introduce in Section 3.³

Given a vector of frequencies $f \in [m]^\ell$, $j \in [m]$ and $i \in [\ell]$, let $\theta_{m,f_i}(j)$ be the angle $2\pi f_i j/m$, and define $\text{cs}_{m,f_i}(j) = [\cos(\theta_{m,f_i}(j)), \sin(\theta_{m,f_i}(j))]^\top$. The representation of j using frequencies $f_0, f_1, \dots, f_{\ell-1}$ is then the vector below.

$$\text{cs}_{m,f}(j) = \begin{bmatrix} \text{cs}_{m,f_0}(j) \\ \text{cs}_{m,f_1}(j) \\ \vdots \\ \text{cs}_{m,f_{\ell-1}}(j) \end{bmatrix}$$

Given a dictionary problem $(k_0, v_0), (k_1, v_1), \dots, (k_{n-1}, v_{n-1}), k^*$, we can now construct the same position input embedding $\mathbf{X} = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{n-1}, \mathbf{x}_n]$ with each $\mathbf{x}_i \in \mathbb{R}^{4\ell}$, as follows. For $j \in [n]$:

$$\mathbf{x}_j = \begin{bmatrix} \text{cs}_{N,f}(\text{idx}(k_j)) \\ \text{cs}_{N,f}(\text{idx}(v_j)) \end{bmatrix}, \text{ and } \mathbf{x}_n = \begin{bmatrix} \text{cs}_{N,f}(\text{idx}(k^*)) \\ \mathbf{0}_{2\ell \times 1} \end{bmatrix}$$

We define the matrices $\mathbf{W}^Q$, $\mathbf{W}^K$, and $\mathbf{W}^V$, relative to f as:

$$\mathbf{W}^Q = \mathbf{W}^K = \begin{bmatrix} \mathbf{I}_{2\ell} & \mathbf{0}_{2\ell \times 2\ell} \end{bmatrix}, \quad \mathbf{W}^V = \begin{bmatrix} \mathbf{0}_{2\ell \times 2\ell} & \mathbf{0}_{2\ell \times 2\ell} \\ \mathbf{0}_{2\ell \times 2\ell} & 2 * \mathbf{I}_{2\ell} \end{bmatrix},$$

3. We note that the original definition of the transformer architecture by [Vaswani et al. \(2017\)](#) uses a position encoding based on multiple frequencies.

Including additional frequencies that increase the minimum gap between the dot product of a matching pair and that of any distinct pair increases the corresponding attention-score gap. This has the effect of magnifying the attention weight assigned to the target key and diminishing the attention weights assigned to distractor keys with similar representations to the target key. This in turn reduces noise in the weighted sum of values, increasing the relative contribution of the correct value to the output of attention.

When representing values from $[m]$ using a single frequency, the choice $f = 1$ guarantees that no two distinct values are mapped to the same representation, and achieves the smallest possible maximum dot product of representations of two distinct values. In the case of multiple frequencies, it is not straightforward to determine which combination of frequencies minimizes the maximum dot product for representations of distinct values. We determined the best combinations of 2 and 4 frequencies for representing $[1000]$ using exhaustive search. In the case of 8 frequencies, we chose the best candidate among 100 million randomly sampled frequency combinations, since exhaustive search became infeasible at this scale.

For $[1000]$, using only the frequency 1 results in a margin of 0.000010 between the attention score of a correct match and the score of a query and the key with the most similar embedding. Using the ideal frequency pair $[1, 88]$ instead increases that margin by almost three orders of magnitude, to 0.007698.

For each vector of frequencies we constructed models with the respective frequency embeddings and the weight matrices introduced in the construction above. The models used residuals but no layer normalization. We then performed inference (no training) on randomly generated dictionaries of size 100 and a choice of 1000 keys and values. We gradually decreased the softmax temperature and recorded the accuracy.

4.2. Results

As shown in Figure 2, we found that with sufficiently low temperatures, all models achieved 100% accuracy. Models using more frequencies and dimensions for their embeddings succeeded with temperatures closer to 1. Experiments with a slightly modified embedding yielded similar results for models with layer normalization.

5. Consecutive Positions: Theoretical Bounds

When the keys k_i and values v_i are stored in consecutive positions, information about a key and its corresponding value requires coordination between specific pairs of different input positions. It is clear that there must be a position embedding to enable such coordination. In this section we prove asymptotically matching lower and upper bounds of $\Theta(N \log N)$ on the c-size of a family of 1-layer transformers to perform dictionary lookup for a vocabulary of cardinality N in the consecutive positions case.

5.1. Consecutive Positions: A 1-Layer Lower Bound of $\Omega(N \log N)$

Sanford et al. (2024) use communication complexity to prove a worst-case lower bound on the c-size of any family of 1-layer transformers to perform the induction heads task. They prove their lower bound under the assumption that the transformer “uses p bits of precision”, stated as follows: “... the outputs of all embedding functions ... and self-attention heads

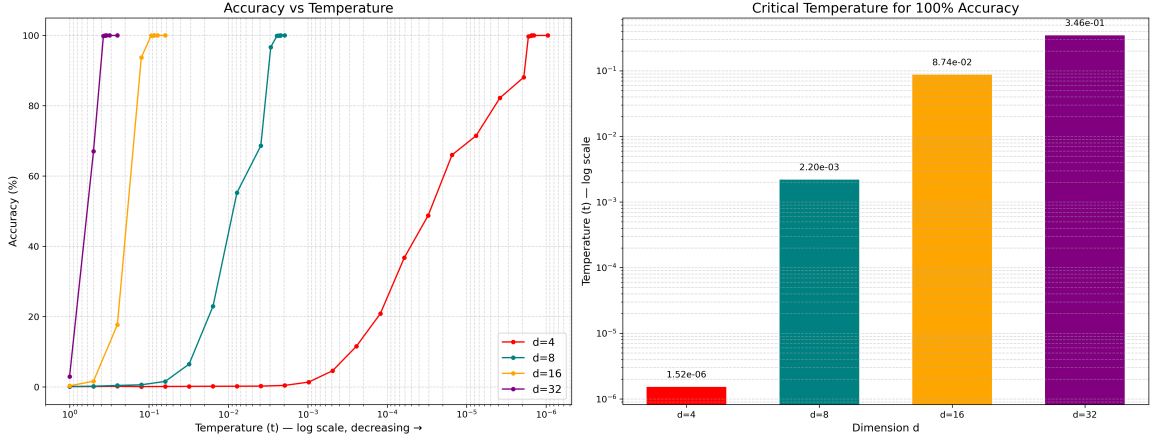


Figure 2: Temperatures required for models with dimensions 4, 8, 16, 32 to achieve 100% accuracy using the theoretical construction. The embeddings are based on 1, 2, 4, and 8 frequencies, respectively.

may be rounded to rational numbers with at most p bits of precision without changing the behavior of the mapping [defined by the transformer].” We refer to this assumption as the *p-bit rational precision assumption*. We adapt their argument to prove the following theorem.

Theorem 3 Suppose T is a 1-layer transformer satisfying the *p-bit rational precision assumption*, with h self-attention heads and embedding dimension d . If T correctly performs the dictionary lookup task for a vocabulary $\mathcal{V}$ of cardinality N in the consecutive positions case, then $hdp \geq (1/3)(N \log N)$.

Note that this result applies to any 1-layer transformer satisfying the assumptions, including those with arbitrary position encodings and softmax attention or average hard attention. In particular, the claim about arbitrary position encodings is restricted to encodings whose outputs satisfy the *p-bit rational precision assumption*. The proof is in §C.

5.2. Consecutive Positions: A 1-Layer Softmax Attention Upper Bound of $O(N \log N)$

The general construction for memorization given by [Kajitsuka and Sato \(2024\)](#) implies that there is a transformer with one softmax attention layer that performs the dictionary lookup task correctly, but the upper bound on its c-size derived from their general result is at least N^N . We prove the following stronger result in the specific case of dictionary lookup.

Theorem 4 For any positive integer N , there exists a soft attention transformer with a suitable position embedding that correctly performs dictionary lookup for a vocabulary of cardinality N in the consecutive positions case. The transformer has one attention layer with softmax attention, one head, embedding dimension $2N + 2$, $O(\log N)$ bits of precision,

and a constant number of position-wise feedforward layers with a total of $N^{O(1)}$ parameters. The c-size is $hdp = O(N \log N)$.

Proof Let $d = 2N + 2$ be the embedding dimension. Suppose the input sequence of tokens representing a dictionary lookup problem is $k_0, v_0, \dots, k_{n-1}, v_{n-1}, k^*$ for some $n \leq N$. Let $r_0, r_1, \dots, r_{2n}$ be the sequence of indices of these tokens, so $r_0 = \text{idx}(k_0)$, $r_1 = \text{idx}(v_0)$ and so on, concluding with $r_{2n} = \text{idx}(k^*)$. We describe a construction for a 1-layer transformer that copies a representation of this sequence of indices into the first $2n + 1$ dimensions of its activation vector, so that the position-wise feedforward layer at position $2n$ has access to the entire input sequence. The position embedding maps position $i \in [2n + 1]$ to the one-hot vector in $\mathbb{R}^d$ that has a single 1 at index i . The word embedding maps $\tau \in \mathcal{V}$ to the vector $[0, 0, \dots, 0, \ln(\text{idx}(\tau) + 1)]^\top \in \mathbb{R}^d$.

We set $d_{\text{key}} = 1$, $\mathbf{W}^K = [0, \dots, 0, 1]$, and $\mathbf{W}^Q = [1, \dots, 1, 0]$. Then for all positions i and j , $\mathbf{W}^K \mathbf{x}_j = \ln(r_j + 1)$, $\mathbf{W}^Q \mathbf{x}_i = 1$. Considering the position of the target key, $2n$, its attention score for position j is $\ln(r_j + 1)$, which exponentiates to $r_j + 1$. Thus the attention weight is $(r_j + 1)/S$, where $S = \sum_{k=0}^{2n} (r_k + 1)$. The value matrix extracts the first $d - 1$ coordinates of $\mathbf{x}_j$ and sets the last coordinate to 0. The result of the attention computation at position $2n$ is $[(r_0 + 1)/S, (r_1 + 1)/S, \dots, (r_{2n} + 1)/S, 0, \dots, 0]^\top \in \mathbb{R}^d$. Having a representation of the whole input sequence, a constant number of position-wise feedforward layers with $N^{O(1)}$ parameters and $O(\log N)$ bits of precision suffice for the computation. The c-size is $hdp = O(N \log N)$. ■

5.3. Consecutive Positions: A Concise Family of 2-Layer Transformers

If we allow our transformers to have *two* layers, there is a concise family of transformers that performs dictionary lookup in the consecutive positions case.

Theorem 5 *For any positive integer N , there exists a 2-layer transformer with a suitable position embedding that performs dictionary lookup for a vocabulary $\mathcal{V}$ of cardinality N in the consecutive positions case. The transformer has embedding dimension 7, average hard attention, one head, $O(\log N)$ bits of precision, residuals, feedforward layers, and no layernorm. The c-size is $hdp = O(\log N)$.*

The construction is similar to that for induction heads: using the position embedding, the key information is copied to the same position as the value information, and the 1-layer solution is adapted to produce the correct answer. The proof is in §D.

6. Training Experiments

The preceding theoretical results concern expressivity, which is a necessary but not sufficient condition for trainability. Here we test the ability of small transformers with 1 layer to be trained to perform dictionary lookup in the same position and consecutive positions cases. These experiments are designed to provide a sanity check for the qualitative distinction between the two cases.

Experimental Setup: In all experiments, we use n distinct keys and n possibly non-distinct corresponding values, both taken from $[N]$. We test cases where the token embedding is learned, as well as cases where we initialize models with custom token embeddings taken from our constructions. For each input sequence, the keys are a random sequence of n distinct elements of $[N]$ and the target key is a random element of that sequence. The inputs are presented to the transformer in two principal ways:

Same position cases: Each input position simultaneously stores a key-value pair by concatenating their token embeddings into a single vector in $\mathbb{R}^d$. The transformer thus receives both the key and its corresponding value information at each position.

Consecutive position cases: Keys and their corresponding values are embedded in $\mathbb{R}^d$ and stored in adjacent but separate input positions. Specifically, keys occupy even-indexed positions, while their associated values occupy subsequent odd-indexed positions. In each case, $d_{\text{hid}} = d$.

Encoder-only transformers with one attention layer were trained for varying embedding dimensions and dictionary sizes, using a single attention head. All models were trained using the Adam optimizer (learning rate 0.001), with 10000 batches of 256 samples of randomly generated inputs across five independent runs with different random seeds. A run stops early if its loss reaches < 0.01 . We measure performance as accuracy in predicting the correct dictionary lookup for each input across 100 test batches. Residual connections are applied consistently across all attention and feed-forward layers. We did not use layer normalization or dropout.

6.1. Results

Same Position: We trained models of dimensions $d = 4$, $d = 8$, $d = 16$, and $d = 32$ with both random initialization and using the custom frequency-based embeddings and weight matrices introduced in §4. For the custom initialization, $d = 4, 8, 16, 32$ correspond to 1, 2, 4, 8 frequencies, respectively. The construction in Thm. 1 uses only $d = 4$; the larger custom models test whether additional frequencies improve trainability, while the larger randomly initialized models test whether additional embedding dimensions improve trainability. Though $d = 4$ is sufficient for theoretical expressivity, all trained models of this size performed approximately at chance. For all other embedding dimensions, models initialized with custom weights and embeddings achieved above 99% accuracy after training. Randomly initialized models struggled in the $d = 8$ case but matched the custom models for $d=16$ and $d=32$. Results for dictionaries of size 100 and vocabulary of size 1000 are shown in Table 1.

Initialization	$d = 4$		$d = 8$		$d = 16$		$d = 32$	
random	1.25	0.37	61.73	1.05	99.77	0.04	99.94	0.03
custom	1.84	1.46	99.86	0.10	99.99	0.00	99.99	0.02

Table 1: Accuracy (mean and SD) for same position dictionaries with $n = 100$, $N = 1000$, using one attention layer. There is no position embedding.

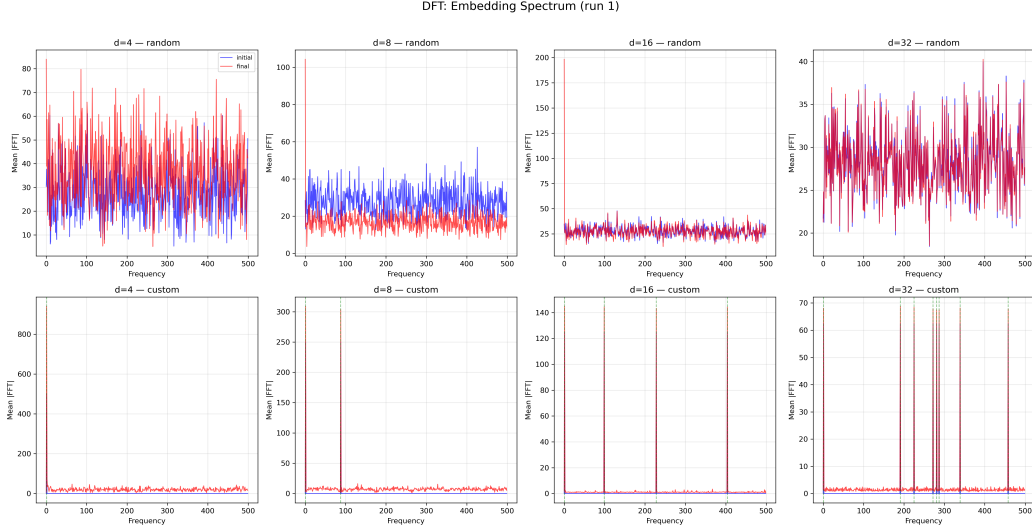


Figure 3: Discrete Fourier transform analysis of initialized (blue) and learned (red) embeddings. The top row corresponds to models with random initialization with different embedding dimension, while the bottom row represents models with custom initializations. Note the dramatic spike at frequency 1 in the learned embeddings with random initialization for $d = 8, 16$.

We explored the degree to which learned models converged to a representation that makes use of frequency-based encodings. Using the discrete Fourier transform (Cormen et al., 2001) to analyze the set of embeddings revealed that models initialized using custom embeddings with varying numbers of frequencies hew closely to these representations throughout training (Table 3 bottom). In contrast, randomly initialized models for $d = 8, 16$ come to have clear peaks at frequency 1 but no others, (Table 3 top). At dimension 32 the randomly initialized model shows no heightened sensitivity to one particular frequency over the others, suggesting that at that point the embedding space has become large enough for other strategies to come online.

Consecutive Positions: Table 2 shows results for consecutive positions, with each row corresponding to a different dictionary and vocabulary size, n and N respectively. Recall that, under the p -bit rational precision assumption, no concise family of 1-layer transformers performs this task for all N . We find that 1-layer transformers exhibited significant struggles in learning dictionary lookup, especially for larger n and N . In the case of $n = 10, N = 20$ larger d were associated with higher accuracy.

7. Conclusion

We have considered the capacity of different abstract models of transformers to perform two variants of a dictionary lookup task, a fundamental building block of many algorithms. In

n	N	$d = 4$		$d = 8$		$d = 16$		$d = 32$	
10	20	13.64	0.30	18.07	0.55	21.25	0.17	21.71	0.09
20	200	3.97	0.30	3.43	0.25	3.62	0.12	4.92	0.16
100	1000	0.45	0.04	0.57	0.04	0.58	0.03	0.47	0.03

Table 2: Accuracy (mean and SD) for consecutive position dictionaries, across varying n and N , using one attention layer. The position embedding is learned.

the first variant, keys and values are represented in a shared input position, while in the second variant, each value is stored in the position following that of the associated key.

We have shown that the first variant of this task can be computed by concise 1-layer transformers; we provide a construction for a family of concise 1-layer transformers that accomplishes the task using average hard attention, one head, embedding dimension 4, no position embedding and $O(\log N)$ bits of precision, where N is the cardinality of the key and value vocabulary. Our construction uses no residuals, layer norms, or feedforward layers, but these can be incorporated using techniques described by [Yang et al. \(2025\)](#). To extend this result to transformers that use softmax attention, we explore the efficacy of adding a temperature parameter to the softmax computation. We experimentally determine the temperature value necessary to succeed on the task with a frequency-based embedding, which varies by model dimensionality.

For the second variant of the dictionary lookup task, where keys and values are stored in consecutive positions, we prove that families of concise 1-layer transformers are not able to perform the task under the assumption of p -bit rational precision. We demonstrate however that a family of concise 2-layer transformers can perform this task, using constant embedding dimension 7, average hard attention, one head, residuals, feedforward layers, no layernorm, and $O(\log N)$ bits of precision, where N is the cardinality of the key and value vocabulary.

Having the capacity to perform a computation does not, of course, ensure that such a task can be feasibly learned. We therefore briefly explore the viability of learning in 1-layer transformers. We find that in the first variant, transformers are indeed able to achieve ceiling performance, so long as the model dimension is sufficiently large. Attempts to train 1-layer transformers on the consecutive position dictionary lookup task do not succeed, in accordance with the theoretical predictions.

Returning to the first variant of the task, we find some benefit in initializing the model with the custom embeddings we define in our construction that use frequency-based encodings: in this case, models with smaller dimensionality ($d = 8$) can achieve success. We then analyzed the learned representations of custom and randomly initialized models using the discrete Fourier transform. Custom initialized retain their preference for the use of frequency-based representations across learning. In contrast, randomly initialized transformers learn embeddings with a significant peak at frequency 1, but no other frequencies. This suggests that inductive biases of these models, when randomly initialized, do not favor the kind of frequency-based representation we hypothesize and which permits more efficient generalization. We leave it for future work to shed light on the nature of the representations that are preferred in this context, and how to imbue transformer models with a bias in favor of these representations.

Acknowledgments

This work includes and builds on results from our earlier, unpublished work (Strobl et al., 2025). The authors thank David Chiang, Alexander Kozachinskiy, Binghui Peng, Clayton Sanford, and Andy Yang for their generous help. We are grateful to members of the Computational Linguistics at Yale (CLAY) lab for their useful commentary. We thank the Yale Center for Research Computing for guidance and assistance in computation run on the Bouchet cluster. Lena Strobl was supported by the Knut and Alice Wallenberg Foundation (KAW 2024.0443).

AI Statement

AI coding assistance was used to write code for this paper, including for the generation of figures and tables. All outputs were reviewed and verified by the authors.

References

- Lijie Chen, Binghui Peng, and Hongxun Wu. Theoretical limitations of multi-layer transformer, 2024. URL <https://arxiv.org/abs/2412.02975>.
- David Chiang and Peter Cholak. Overcoming a theoretical limitation of self-attention. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7654–7664, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.527. URL <https://aclanthology.org/2022.acl-long.527/>.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. The MIT Press and McGraw-Hill Book Company, Cambridge, Massachusetts and New York, New York, 2nd edition, 2001.
- Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. A mathematical framework for transformer circuits, 2021. URL <https://transformer-circuits.pub/2021/framework/index.html>.
- Tokio Kajitsuka and Issei Sato. Are transformers with one layer self-attention using low-rank weight matrices universal approximators? In *Proceedings of ICLR*, 2024. URL <https://arxiv.org/abs/2307.14023>.
- Juno Kim and Taiji Suzuki. Transformers provably solve parity efficiently with chain of thought, 2025. URL <https://arxiv.org/abs/2410.08633>.
- Alexander Kozachinskiy, Felipe Urrutia, Hector Jimenez, Tomasz Steifer, Germán Pizarro, Matías Fuentes, Francisco Meza, Cristian B. Calderon, and Cristóbal Rojas. Strassen attention: Unlocking compositional abilities in transformers based on a new lower bound method, 2025. URL <https://arxiv.org/abs/2501.19215>.

- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, Tom Conerly, Dawn Drain, Deep Ganguli, Zac Hatfield-Dodds, Danny Hernandez, Scott Johnston, Andy Jones, Jackson Kernion, Liane Lovitt, Kamal Ndousse, Dario Amodei, Tom Brown, Jack Clark, Jared Kaplan, Sam McCandlish, and Chris Olah. In-context learning and induction heads, 2022. URL <https://arxiv.org/abs/2209.11895>.
- Binghui Peng, Srini Narayanan, and Christos Papadimitriou. On limitations of the transformer architecture, 2024. URL <https://arxiv.org/abs/2402.08164>.
- Clayton Sanford, Daniel Hsu, and Matus Telgarsky. Representational strengths and limitations of transformers, 2023. URL <https://arxiv.org/abs/2306.02896>.
- Clayton Sanford, Daniel Hsu, and Matus Telgarsky. One-layer transformers fail to solve the induction heads task, 2024. URL <https://arxiv.org/abs/2408.14332>.
- Lena Strobl, William Merrill, Gail Weiss, David Chiang, and Dana Angluin. What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561, 05 2024. ISSN 2307-387X. doi: 10.1162/tacl_a_00663. URL https://doi.org/10.1162/tacl_a_00663.
- Lena Strobl, Dana Angluin, and Robert Frank. Concise one-layer transformers can do function evaluation (sometimes), 2025. URL <https://arxiv.org/abs/2503.22076>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017. URL https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf.
- Andy Yang, Christopher Watson, Anton Xue, Satwik Bhattamishra, Jose Larena, William Merrill, Emile Dos Santos Ferreira, Anej Svete, and David Chiang. The transformer cookbook, 2025. URL <https://arxiv.org/abs/2510.00368>.
- Andy Yang, Lena Strobl, David Chiang, and Dana Angluin. Simulating hard attention using soft attention. *Transactions of the Association for Computational Linguistics*, 14:147–166, 2026.
- Chulhee Yun, Srinadh Bhojanapalli, Ankit Singh Rawat, Sashank J. Reddi, and Sanjiv Kumar. Are transformers universal approximators of sequence-to-sequence functions? In *Proceedings of ICLR*, 2020. URL <https://arxiv.org/abs/1912.10077>.

Appendix A. Example of the Representation of Integers from $[m]$

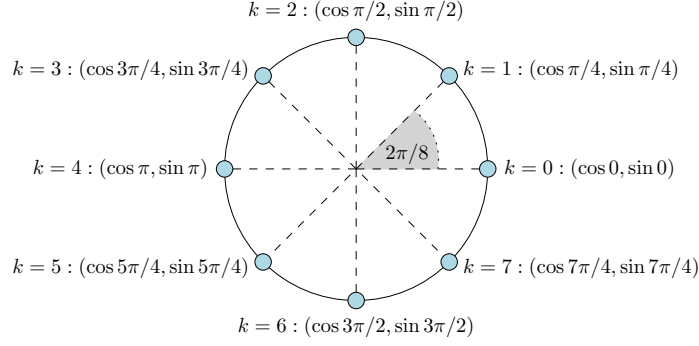


Figure 4: Each integer $k \in [8]$ is mapped to a point on the unit circle using the transformation $\text{cs}_8(k) = [\cos(\frac{2\pi k}{8}), \sin(\frac{2\pi k}{8})]$. E.g., $k = 0$ maps to $(1, 0)$, $k = 2$ maps to $(0, 1)$, and so on. Dashed radial lines show the angles between consecutive points.

Appendix B. Example of the Construction in [Thm. 1](#)

Example 1 (Example for [Thm. 1](#)) In [Thm. 1](#), we prove that for any positive integer N , there exists a 1-layer transformer that performs dictionary lookup for a vocabulary of cardinality N in the same position case. The transformer has average hard attention, one head, an embedding dimension of 4, and $O(\log N)$ bits of precision. Here we describe an example of the construction for $N = 4$ and a dictionary of length 4.

Representation of Integers. Each integer $k \in [4]$ is encoded using the mapping

$$\text{cs}_4(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \text{cs}_4(1) = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \text{cs}_4(2) = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \text{cs}_4(3) = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

This encoding corresponds to mapping the integer k to a point on the unit circle with angle $\theta_4(k) = \frac{2\pi k}{4}$.

Input Construction. Consider a dictionary consisting of the following key-value pairs:

$$(k_0, v_0) = (0, 2), \quad (k_1, v_1) = (2, 3), \quad (k_2, v_2) = (1, 2), \quad (k_3, v_3) = (3, 1).$$

Let the target key be $k^* = 2$, so the correct output is 3. The input sequence is constructed as follows. For $j \in \{0, 1, 2, 3\}$, define

$$\mathbf{x}_j = \begin{bmatrix} \text{cs}_4(k_j) \\ \text{cs}_4(v_j) \end{bmatrix}.$$

Thus,

$$\mathbf{x}_0 = \begin{bmatrix} \text{cs}_4(0) \\ \text{cs}_4(2) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ -1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_1 = \begin{bmatrix} \text{cs}_4(2) \\ \text{cs}_4(3) \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 0 \\ -1 \end{bmatrix},$$

$$\mathbf{x}_2 = \begin{bmatrix} \text{cs}_4(1) \\ \text{cs}_4(2) \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ -1 \\ 0 \end{bmatrix}, \quad \mathbf{x}_3 = \begin{bmatrix} \text{cs}_4(3) \\ \text{cs}_4(1) \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 1 \end{bmatrix}.$$

An additional input vector encodes the target key:

$$\mathbf{x}_4 = \begin{bmatrix} \text{cs}_4(i^*) \\ \mathbf{0}_{2 \times 1} \end{bmatrix} = \begin{bmatrix} \text{cs}_4(2) \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Linear Transformations. The transformer uses the following weight matrices:

$$\mathbf{W}^Q = [\mathbf{I}_2 \quad \mathbf{0}], \quad \mathbf{W}^K = [\mathbf{I}_2 \quad \mathbf{0}], \quad \mathbf{W}^V = \begin{bmatrix} \mathbf{0}_{2 \times 2} & \mathbf{0}_{2 \times 2} \\ \mathbf{0}_{2 \times 2} & 2 * \mathbf{I}_2 \end{bmatrix}.$$

Thus, the query at the target position (position 4) is computed as

$$\mathbf{q}_4 = \mathbf{W}^Q \mathbf{x}_4 = \begin{bmatrix} -1 \\ 0 \end{bmatrix},$$

and for positions 0 through 4, the key vectors are

$$\mathbf{k}_0 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad \mathbf{k}_1 = \begin{bmatrix} -1 \\ 0 \end{bmatrix}, \quad \mathbf{k}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad \mathbf{k}_3 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}, \quad \mathbf{k}_4 = \begin{bmatrix} -1 \\ 0 \end{bmatrix},$$

with corresponding value vectors

$$\mathbf{v}_0 = \begin{bmatrix} 0 \\ 0 \\ -2 \\ 0 \end{bmatrix}, \quad \mathbf{v}_1 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -2 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 0 \\ 0 \\ -2 \\ 0 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 2 \end{bmatrix}, \quad \mathbf{v}_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}.$$

Attention Score Computation. The attention score between the query at position 4 and a key at position j is given by

$$s_j = \frac{q_4^\top k_j}{\sqrt{2}},$$

which yields:

$$s_0 = \frac{-1}{\sqrt{2}}, \quad s_1 = \frac{1}{\sqrt{2}}, \quad s_2 = \frac{0}{\sqrt{2}}, \quad s_3 = \frac{0}{\sqrt{2}}, \quad s_4 = \frac{1}{\sqrt{2}}.$$

Average Hard Attention and Output. Using average hard attention, the transformer assigns attention weight $1/2$ to the two positions with the maximum score of $\frac{1}{\sqrt{2}}$ and 0 to the other positions, that is

$$\alpha_1 = \alpha_4 = \frac{1}{2} \quad \text{and} \quad \alpha_j = 0 \text{ for } j \in \{0, 2, 3\}.$$

Thus, the output of the attention layer is

$$\mathbf{h} = \frac{1}{2}\mathbf{v}_1 + \frac{1}{2}\mathbf{v}_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1 \end{bmatrix}.$$

Unembedding. Finally, an unembedding matrix $\mathbf{U}$ is used to convert $\mathbf{h}$ into the final output. We have the following.

$$\hat{\mathbf{v}} = \mathbf{U}\mathbf{h} = \begin{bmatrix} 0 & 0 & \cos 0 & \sin 0 \\ 0 & 0 & \cos \frac{\pi}{2} & \sin \frac{\pi}{2} \\ 0 & 0 & \cos \pi & \sin \pi \\ 0 & 0 & \cos \frac{3\pi}{2} & \sin \frac{3\pi}{2} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ -1 \end{bmatrix} = \begin{bmatrix} 0 \\ -1 \\ 0 \\ 1 \end{bmatrix}.$$

Because the maximum entry in $\hat{\mathbf{v}}$ is at index 3, the transformer outputs 3, which matches the correct dictionary lookup output of 3.

Appendix C. Proof of [Thm. 3](#)

We define the *function evaluation task with domain $[N]$* , which is a specialization of the dictionary lookup task such that the vocabulary is $[N]$ and every dictionary has length N . For such inputs, the dictionary can be viewed as representing a finite function $f : [N] \rightarrow [N]$, and the problem of dictionary lookup can be viewed as evaluating f on a particular input k^* . Note that there are N^N such functions.

We consider the following two-party communication complexity problem, which we term *evaluate f with domain $[N]$ on 0*. Let $k_0, \dots, k_{N-1}$ be a permutation of $[N]$ and let $v_0, \dots, v_{N-1}$ be any sequence of elements from $[N]$, where these two sequences represent the function $f(k_i) = v_i$. Alice knows the sequence $v_0, \dots, v_{N-1}$ and Bob knows the sequence $k_0, \dots, k_{N-1}$. Alice sends a message to Bob, after which he is required to output the unique v_j such that $k_j = 0$.

Lemma 6 *In any communication protocol for the problem of evaluating f with domain $[N]$ on 0 in which Bob always outputs the correct answer, Alice’s message must sometimes contain at least $N \log_2 N$ bits.*

Proof If not, there will be two different sequences $v_0, \dots, v_{N-1}$ and $v'_0, \dots, v'_{N-1}$ for which Alice sends the same message. There is some position j such that $v_j \neq v'_j$. Thus, when $k_j = 0$, Bob cannot produce the correct (different) answers for both sequences. ■

[Sanford et al. \(2024\)](#) give a careful analysis of bit precision in a 1-layer transformer computation that shows the following.

Lemma 7 *Suppose T is a 1-layer transformer satisfying the p -bit rational precision assumption, with h self-attention heads and embedding dimension d . If T correctly performs the function evaluation task with domain $[N]$ in the consecutive positions case, then there is a communication protocol for the problem of evaluating f with domain $[N]$ on 0 in which the message sent by Alice has length at most $3hdp$.*

Thm. 3 follows from these two lemmas because if a transformer T correctly performs the dictionary lookup task for a vocabulary of cardinality N in the consecutive positions case, then T also performs the function evaluation task with domain $[N]$ in the consecutive positions case.

Appendix D. Proof of **Thm. 5**

Proof The vocabulary is $\mathcal{V}$ with $N = |\mathcal{V}|$. The sequence of inputs is $k_0, v_0, \dots, k_{n-1}, v_{n-1}, k^*$. We use two layers with average hard attention. The first layer copies k_r from position $2r$ into position $2r + 1$, so that position $2r + 1$ contains both k_r and v_r . The second layer applies a version of the 1-layer construction from **Thm. 1** for the same position case. It ignores the even positions and extracts v_r such that $k_r = k^*$.

The token and position embeddings are defined as follows, for $\tau \in \mathcal{V}$ and $r \in [n + 1]$.

$$\begin{aligned} \mathbf{w}(\tau) &= [0, 0, \text{cs}_N(\text{idx}(\tau)), 0, 0, 0]^\top \\ \mathbf{p}(2r) &= [0, 0, 0, 0, \text{cs}_{N+1}(r), 1]^\top \\ \mathbf{p}(2r + 1) &= [0, 0, 0, 0, \text{cs}_{N+1}(r), 0]^\top. \end{aligned}$$

Note that the position embedding for k_r and v_r are the same in coordinates 4 and 5 but different in coordinate 6. The token embedding and the position embedding use different representations of integers, cs_N and cs_{N+1} . Thus the input vectors $\mathbf{x}_i$ are as follows.

If $r \in [n]$ and $i = 2r$ then

$$\mathbf{x}_i = [0, 0, \text{cs}_N(\text{idx}(k_r)), \text{cs}_{N+1}(r), 1]^\top.$$

If $r \in [n]$ and $i = 2r + 1$ then

$$\mathbf{x}_i = [0, 0, \text{cs}_N(\text{idx}(v_r)), \text{cs}_{N+1}(r), 0]^\top.$$

For $i = 2n$,

$$\mathbf{x}_i = [0, 0, \text{cs}_N(\text{idx}(k^*)), \text{cs}_{N+1}(n), 1]^\top.$$

Layer 1. Matrices $\mathbf{W}_1^Q, \mathbf{W}_1^K \in \mathbb{R}^{3 \times 7}$ both map $\mathbf{x}_i$ to the vector $[\text{cs}_{N+1}(r), 0]^\top$, where $i = 2r$ or $i = 2r + 1$, by extracting coordinates 4 and 5 of $\mathbf{x}_i$. For $r \in [n]$, the attention scores for positions $i = 2r$ and $i = 2r + 1$ are thus maximized at positions $j = 2r$ and $j = 2r + 1$, each of which gets attention weight $1/2$. The attention score for position $i = 2n$ is uniquely maximized at position $j = 2n$, which gets attention weight 1. The matrix $\mathbf{W}_1^V \in \mathbb{R}^{7 \times 7}$ copies the two values from dimensions 2 and 3 of the attended-to positions j into the dimensions 0 and 1 of the result, and sets the other dimensions to 0. Thus the results $\mathbf{y}_i$ of the first attention layer after the residual is added are as follows.

If $r \in [n]$ and $i = 2r$ then

$$\mathbf{y}_i = \left[\frac{1}{2} \text{cs}_N(\text{idx}(k_r)) + \frac{1}{2} \text{cs}_N(\text{idx}(v_r)), \text{cs}_N(\text{idx}(k_r)), \text{cs}_{N+1}(r), 1 \right]^\top.$$

If $r \in [n]$ and $i = 2r + 1$ then

$$\mathbf{y}_i = \left[\frac{1}{2} \text{cs}_N(\text{idx}(k_r)) + \frac{1}{2} \text{cs}_N(\text{idx}(v_r)), \text{cs}_N(\text{idx}(v_r)), \text{cs}_{N+1}(r), 0 \right]^\top.$$

For $i = 2n$,

$$\mathbf{y}_i = [\text{cs}_N(\text{idx}(k^*)), \text{cs}_N(\text{idx}(k^*)), \text{cs}_{N+1}(n), 1]^\top.$$

The first feedforward layer subtracts the values in coordinates 2 and 3 from twice the values in coordinates 0 and 1. Thus the inputs $\mathbf{x}'_i$ to the second attention layer are as follows.

If $r \in [n]$ and $i = 2r$ then

$$\mathbf{x}'_i = [\text{cs}_N(\text{idx}(v_r)), \text{cs}_N(\text{idx}(k_r)), \text{cs}_{N+1}(r), 1]^\top.$$

If $r \in [n]$ and $i = 2r + 1$ then

$$\mathbf{x}'_i = [\text{cs}_N(\text{idx}(k_r)), \text{cs}_N(\text{idx}(v_r)), \text{cs}_{N+1}(r), 0]^\top.$$

For $i = 2n$,

$$\mathbf{x}'_i = [\text{cs}_N(\text{idx}(k^*)), \text{cs}_N(\text{idx}(k^*)), \text{cs}_{N+1}(n), 1]^\top.$$

Layer 2. We use the approach from [Thm. 1](#) for the same position case, making use of the even/odd portion of the position embedding. The matrix $\mathbf{W}_2^K \in \mathbb{R}^{3 \times 7}$ copies dimensions 0, 1, and 6 from $\mathbf{x}'_i$. Thus, for $r \in [n]$ and position $j = 2r$, the key vector is

$$\mathbf{k}_j = \begin{bmatrix} \text{cs}_N(\text{idx}(v_r)) \\ 1 \end{bmatrix},$$

for $r \in [n]$ and position $j = 2r + 1$, the key vector is

$$\mathbf{k}_j = \begin{bmatrix} \text{cs}_N(\text{idx}(k_r)) \\ 0 \end{bmatrix},$$

and for $j = 2n$, the key vector is

$$\mathbf{k}_{2n} = \begin{bmatrix} \text{cs}_N(\text{idx}(k^*)) \\ 1 \end{bmatrix}.$$

The matrix $\mathbf{W}_2^Q \in \mathbb{R}^{3 \times 7}$ copies dimensions 0, 1 of $\mathbf{x}'_i$ and multiplies dimension 6 by -1 . Thus, for position $2n$, the query is

$$\mathbf{q}_{2n} = \begin{bmatrix} \text{cs}_N(\text{idx}(k^*)) \\ -1 \end{bmatrix}.$$

The attention score for position $2n$ is uniquely maximized at $j^* = 2r + 1$ such that $k_r = k^*$. The matrix $\mathbf{W}_2^V$ copies dimensions 2 and 3 of the attended-to position j^* and sets the other dimensions to 0, yielding the desired value $[\text{cs}_N(\text{idx}(v_r)), 0, 0, 0, 0, 0]^\top$. The unembedding matrix $\mathbf{U}$ consists of N rows of the form $[\text{cs}_N(i), 0, 0, 0, 0, 0]$ for $i \in [N]$, which gives the correct output v_r . $\blacksquare$